Proyecto 1 – Juego 2D de Plataformas

Curso: Introducción a la Programación

Grupo: 01

Estudiante: Fabián Gerardo Cambronero Núñez

Carné: 2026079420

Profesores: Diego Mora Rojas y Jeff Schmidt Peralta

Fecha de entrega: 07/05/26

Periodo: 1 Semestre 2026

1. **Objetivo del proyecto**

El objetivo del proyecto es crear un juego de plataformas en 2D desarrollado en Python y la biblioteca Tkinter donde el jugador debe poder controlar un personaje desde un punto de inicio hasta una meta final. Durante el recorrido el jugador debe moverse por el escenario, saltar entre plataformas, utilizar escaleras y evitar enemigos o trampas que pueden provocar la derrota. El juego debe permitir al usuario crear sus propios mapas y retos con un modo editor, dependiendo de los elementos que el usuario coloque el sistema de puntaje aumenta o disminuye.

Cada vez que un jugador logra el éxito y llega a la meta el resultado debe ser guardado, el programa mostrará los cinco mejores puntajes de personas que han logrado llegar a la meta.

2. **Planteamiento general de la solución**

La solución del proyecto se desarrolló utilizando Python y la biblioteca Tkinter para la creación de la interfaz gráfica. El programa se organiza en diferentes ventanas, por ejemplo el menú principal, la ventana del juego, la pantalla de resultado y la pantalla de rankings.

El escenario del juego se representa mediante una matriz donde cada posición corresponde a un elemento del mapa, por ejemplo algunas posiciones pueden representar bloques, enemigos, escaleras, punto de inicio o la meta. Esta representación nos permite construir visualmente el mapa y también nos facilita identificar colisiones y movimiento del personaje.

El movimiento del personaje se controla mediante eventos del teclado, el personaje puede desplazarse hacia la izquierda y hacia la derecha, saltar para alcanzar plataformas superiores o evitar obstáculos, y utilizar escaleras para subir o bajar. Para el salto se maneja una variable que nos permite simular el movimiento vertical del personaje tomando en cuenta si se encuentra en el aire, en una plataforma o en una escalera.

Los enemigos se colocan en el mapa como obstáculos para aumentar la dificultad del juego, cada enemigo tiene un comportamiento definido, ya sea permanecer en un lugar específico o poder movilizarse en el mapa con un rango determinado. Si el jugador colisiona con un enemigo se activa la condición de derrota.

El modo editor del mapa permite al usuario crear su propio mapa antes de jugar. En este editor se pueden colocar todos los elementos del juego. El juego valida si existe al menos un lugar de inicio y una meta final para asegurar que se pueda jugar correctamente.

3. **Generación y validación del mapa**

El mapa del juego se controla en forma de matriz donde cada posición identifica a un elemento en el mapa. En el modo editor donde el usuario puede crear su propio mapa, el usuario puede colocar el elemento que desea en la casilla que desea, la matriz es capaz de guardar estos elementos para luego poder crearlos con Tkinter en la pantalla.

Antes de poder iniciar la partida, el programa realiza una validación del mapa creado. Primero, se comprueba que exista un punto de inicio y una meta final, ya que estos elementos son necesarios para que el nivel pueda jugarse correctamente. Si alguno de estos elementos no se encuentra colocado, el programa no permite iniciar la partida y solicita al usuario corregir el mapa.

Se valida los elementos colocados debido que cada elemento tiene un puntaje asignado, en caso de los enemigos, al sumar dificultad al nivel el puntaje del nivel incrementa conforme se agregan más enemigos, si se agregan escaleras o plataformas el puntaje disminuye debido que es más fácil poder completar el nivel.

4. **Funciones principales**

a) <u>Dibujar_mapa()</u>

Esta función es la encargada de inicializar la estructura del mapa del juego.
El mapa se representa mediante una matriz donde cada posición puede contener un elemento diferente como bloques, escaleras o enemigos.

Es importante porque permite organizar todos los elementos del escenario de forma ordenada. Gracias a esta función, el programa puede saber qué hay en cada parte del mapa y luego dibujarlo en la ventana del juego.

b) <u>Mover_horizontal() y mover_vertical()</u>

Estas funciones controlan el movimiento del personaje dentro del escenario. Permiten que el jugador se desplace hacia la izquierda o derecha, suba o baje escaleras y actualice su posición en la pantalla.

Es importante porque permite el movimiento general, sin la implementación de estas funciones no sería capaz el personaje de poder llegar a la meta debido que no podría movilizarse por los obstáculos del mapa.

c) <u>Validar_mapa_editor()</u>

La función es importante porque permite comprobar que el mapa creado por el usuario en el editor cumpla con las condiciones necesarias antes

de iniciar la partida. Esta función verifica que exista un punto de inicio para el jugador y una meta final, ya que sin estos elementos el nivel no tendría una estructura válida ni una condición clara de victoria.

5. Problemas encontrados y solución aplicada

Movimiento

Durante el desarrollo se presentó un problema con el movimiento del jugador, ya que al principio se utilizaba el desplazamiento por casillas dentro de la matriz del mapa, esto hacía que el desplazamiento fuera poco fluido y que el personaje se moviera de una forma extraña.

Para resolverlo se implementó un sistema de movimiento por pixeles, permitiendo que el personaje avanzara de forma más continua dentro del escenario. Esta solución mejoró la jugabilidad y facilitó el manejo del salto.

Mapa creado accidentalmente

Cuando el usuario se encontraba editando un mapa y no colocaba inicio o meta, pero se salía al menú principal, cuando el usuario volvía a ingresar al juego, el mapa creaba solo el inicio y no se podía completar el nivel debido que no existía una meta existente.

Para resolver este inconveniente se agregó una ventana emergente que le avisará al usuario que no puede cerrar el editor de mapas sin haber definido un mapa válido para poder jugar.

Los enemigos móviles no aparecían o no se movían bien

Al colocar más de un enemigo móvil en el mapa, algunos no aparecían correctamente o se mostraban como bloques grises.

Se cambiaron los enemigos móviles para manejarlos con listas separadas de posición y dirección. Así se pudo recorrer la matriz, detectar todos los enemigos móviles y moverlos de forma independiente.

La tecla R reiniciaba el juego mientras se escribía el nombre

Al escribir el nombre del jugador en la pantalla final, si el nombre tenía la letra “R”, el juego se reiniciaba porque esa tecla estaba asignada a reiniciar la pantalla.

Se bloqueó la lectura de teclas del juego mientras la pantalla final estaba activa, permitiendo escribir el nombre sin activar los controles del juego.

El puntaje del mapa editado no se mantenía al jugar

Aunque el editor calculaba el puntaje según enemigos, trampas, bloques y escaleras, al iniciar la partida el puntaje volvía a un valor predeterminado.

Se creó una variable para guardar el puntaje base del mapa. Al guardar el mapa, ese valor se conserva y se usa como puntaje inicial de la partida.

Enemigo móvil y problemas en el aire

Cuando en el modo editor se colocaba un enemigo móvil en el aire el se quedaba estático y eso se puede interpretar como un error del desarrollo.

Para reparar esta anomalía colocamos una validación antes de guardar el mapa. La validación le indica al usuario que no puede colocar enemigos móviles en el aire en caso de que el lo haya colocado en ese lugar.

6. Conclusión

Con la realización de este proyecto se logró desarrollar un juego de plataformas en 2D utilizando Python y Tkinter, se cumplió con los principales requisitos establecidos, El juego permite controlar un personaje, desplazarse por un escenario, saltar entre plataformas, utilizar escaleras, evitar enemigos y trampas, poder llegar a una meta final y registrar los mejores puntajes obtenidos por los jugadores.

Se logró implementar la edición del mapa con el modo editor que permite al usuario crear sus propios mapas, colocar diferentes elementos y validar que el nivel tenga las condiciones necesarias para poder jugarse.

Durante el desarrollo se reforzaron conceptos importantes como el manejo de estructuras de datos, manejo de archivos, validaciones, eventos del teclado e interfaz gráfica. A pesar de los retos que existieron en la implementación de mecánicas nuevas, se logró construir una solución funcional y coherente con los objetivos del proyecto.

Referencias

OpenAI. (2026). *ChatGPT* (GPT-5.5 Thinking) [Modelo de lenguaje grande]. https://chatgpt.com/

Música:

Pixelmaniax. (2025, 21 de mayo). *The wandering Samurai* [Canción]. Pixabay. https://pixabay.com/music/video-games-the-wandering-samurai-344699/

Fondo de pantalla de inicio:

Ramos, D. (s. f.). *Escenario de videojuegos con montañas y flores vector ilustración vectorial* [Vector]. Vecteezy. https://es.vecteezy.com/arte-vectorial/11666869-escenario-de-videojuegos-con-montanas-y-flores-vector-ilustracion-vectorial